



UNIVERSIDADE DE FORTALEZA

ARTHUR WAGNER DE CAR. GON. LEMOS

[2413769] - PROF. MIQUEIAS MAIA

O guia intergaláctico para:

DEPLOY DE JOGOS

(FOR DUMMIES)



SUMÁRIO

- Introdução - 3
- A Unity - 4
- Ferramentas - 5
- Deploy 1 - 7
- Configuração - 8
- Desenvolvimento - 14
- Deploy 2 - 17
- Conclusão - 22



INTRODUÇÃO

Então, você quer publicar seu primeiro jogo...

Não tema, pois esse e-book foi feito especialmente para você. Neste livreto digital, você observará os passos da publicação e manutenção de jogos em primeira mão, utilizando a plataforma de desenvolvimento Unity.



O plano gratuito da Unity oferece acesso ao editor com todas as ferramentas necessárias para criar e desenvolver jogos em 2D e 3D.

Além disso, a Unity oferece diversas ferramentas de auxílio para o desenvolvedor, como: acesso à Unity Asset Store, onde é possível baixar ou comprar ativos como modelos, scripts, e ferramentas; exportação de jogos multiplataforma; pacotes bônus para integração e até deploy em WebGL! (Isso será uma ferramenta secreta que usaremos mais tarde...)

Na Unity, nós usaremos a linguagem de programação C# e o ambiente de desenvolvimento (IDE) Visual Studio Code. Você vai ver mais sobre esses dois logo logo.

Agora, de onde será que veio essa tal Unity?

A UNITY

A Unity foi fundada em 8 de junho de 2005

Foi fundada na Dinamarca, com a ideia de facilitar a vida de quem queria criar jogos, mesmo sem grandes recursos. Desde então, ela se tornou uma das ferramentas mais usadas no mundo para desenvolver jogos e experiências interativas, como realidade aumentada e virtual. Com o lema "crie uma vez, lance em qualquer lugar", a Unity ajudou tanto iniciantes quanto grandes estúdios a transformar suas ideias em realidade, sendo usada não só em jogos, mas também em áreas como educação, arquitetura e cinema.

Curiosidade: Em 2023, a Unity enfrentou uma grande polêmica ao anunciar mudanças na sua política de preços, introduzindo uma taxa por instalação de jogos feitos com ela. A decisão gerou revolta entre desenvolvedores *indie*, resultando em boicotes e protestos. A repercussão foi tão negativa que a empresa precisou voltar atrás e reformular suas práticas para reconquistar a confiança da comunidade.

Entretanto, a Unity ainda se destaca por sua interface intuitiva, compatibilidade com várias plataformas e uma biblioteca gigantesca de recursos na Asset Store, permitindo que desenvolvedores criem projetos de forma prática e eficiente, independentemente do nível de experiência.

Nós sabemos com o que estamos lidando agora. O que mais falta?



FERRAMENTAS

O que nós vamos utilizar?



- A linguagem de programação



- O ambiente de desenvolvimento



- A nossa ferramenta de versionamento



- A nossa ferramenta de *deploy*

FERRAMENTAS

Vamos descobrir mais sobre elas...

- **C#** é a linguagem padrão usada na Unity para programar jogos. Ele é fácil de aprender, mesmo para quem está começando, e ao mesmo tempo poderoso, permitindo criar mecânicas complexas e interações incríveis. Estamos usando C# porque ele é totalmente integrado à Unity, o que torna o desenvolvimento mais rápido e eficiente.
- **Visual Studio Code** (VS Code) é um ambiente de desenvolvimento (IDE) leve e poderoso, perfeito para programar na Unity. Ele é fácil de usar, cheio de recursos úteis, como autocompletar e sugestões de código, e ainda permite adicionar extensões para personalizar a experiência. Usamos o VS Code porque ele é rápido, gratuito e funciona perfeitamente com o C# e a Unity.
- **GitHub** é uma plataforma para organizar e versionar códigos durante o desenvolvimento. Ele permite salvar, compartilhar e acompanhar mudanças no projeto, garantindo que nada se perca e que todos da equipe trabalhem de forma sincronizada. Usaremos o GitHub porque facilita o trabalho em grupo, possibilita reverter alterações problemáticas e mantém um histórico completo do progresso, além de ser muito prático para colaborar em projetos como jogos na Unity.

Só falta aprender sobre mais um...

DEPLOY

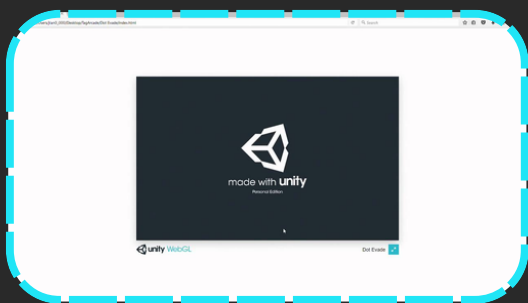
O que exatamente é isso?

Deploy é o processo de colocar um projeto no ar, ou seja, torná-lo acessível para os usuários. No caso de jogos, é quando você finaliza o desenvolvimento e publica o jogo em plataformas como PC, consoles ou dispositivos móveis. Ele é uma etapa essencial para levar o produto do ambiente de desenvolvimento até o público final.

Muitas vezes o deploy não é **final**, ou seja, apenas fazemos um deploy para ter certeza que a compilação, ou seja, todos os detalhes, cenas, código e estrutura do jogo, estão sendo passados para o outro lado com 100% de integridade.

Para o nosso deploy, utilizaremos o **WebGL**, é a tecnologia usada para rodar jogos diretamente no navegador, sem precisar instalar nada. Na Unity, o WebGL permite fazer o deploy do jogo para web de forma simples, transformando seu projeto em algo que qualquer pessoa pode jogar online. É uma ótima opção para alcançar mais pessoas, já que só precisam de um navegador para acessar o jogo.

Agora que já sabemos de tudo, o que falta para começar?

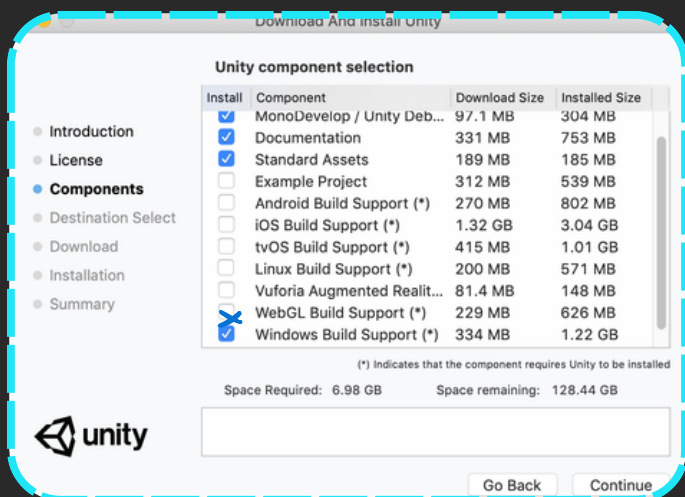


CONFIGURAÇÃO

Passo-a-passo...

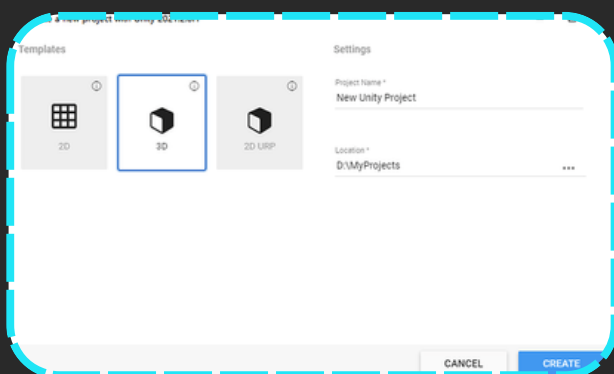
- **Baixar a Unity**

Escolhendo a versão estável mais recente, baixar o plano gratuito da Unity no site oficial após criar sua conta. Ao prosseguir com a instalação, **ter certeza de selecionar a documentação, o Standard Acess e o Pacote WebGL**.



- **Selecionar o Template 2D**

Ao criar um novo projeto, selecione o template "2D" para ajustar o ambiente ao estilo de jogo.



CONFIGURAÇÃO

Passo-a-passo...

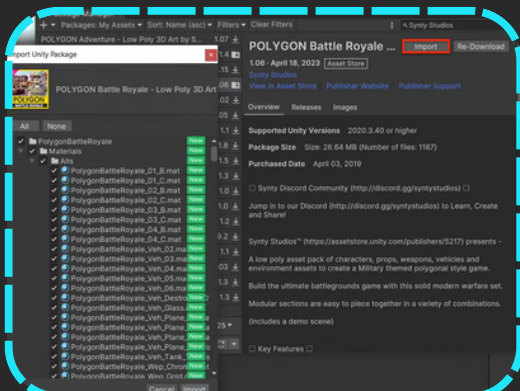
• Organização

A organização na hierarquia de objetos na Unity, pastas, configurações de câmera e outros aspectos é importante para o projeto. Há diversos guias demonstrando como organizar a seu gosto. Um exemplo:



• Importação

Ao criar um novo projeto, é sempre bom ter em mente como você vai importar os objetos. Usar sempre os mesmos tipos e tamanhos é interessante.



CONFIGURAÇÃO

Passo-a-passo...

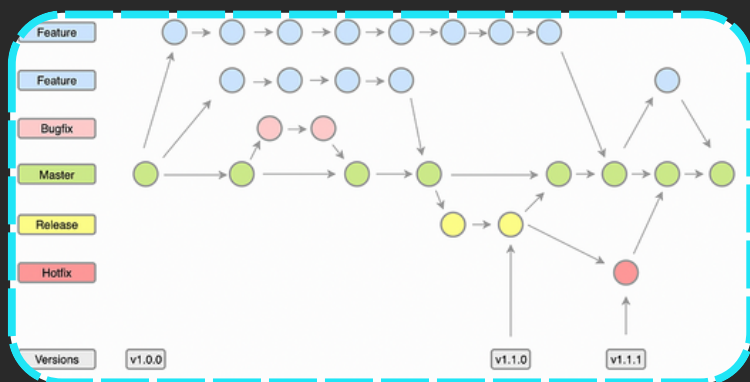
- **GitHub**

Crie uma conta no GitHub. Após criar sua primeira conta, crie seu primeiro repositório e crie ramificações novas que derivam uma da outra no modelo GitFlow. Exemplo a baixo.



- **Utilizando o GitFlow**

Mantenha o seu código fonte, atualizado na Master/Main. Qualquer funcionalidade nova, adicione na Feature/Feat, e passe para a Develop/Dev para revisar. Após revisar, passe para a Main.



CONFIGURAÇÃO

Passo-a-passo...

- **Dependências**

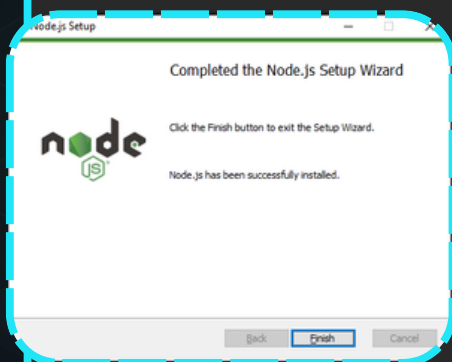
Instalação do Node.Js, necessário para utilização do Git.

[Read](#) [Node.js Collab Summit Report](#) →

Executar a JavaScript em Toda Parte

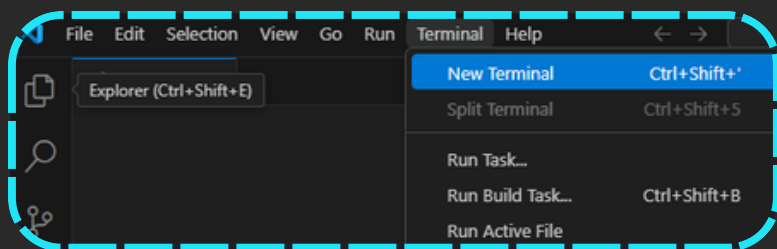
Node.js® é uma ambiente de execução de JavaScript disponível para várias plataformas, de código aberto e gratuita, que permite os programadores criar servidores, aplicações da Web, ferramentas de linha de comando e programas de automação de tarefas.

[Descarregar a Node.js \(LTS\)](#) 



- **Utilizando o Visual Studio Code para GitHub**

Utilizando o terminal do VSCode, vamos fazer a utilização do Node para alterar o GitHub.



- Clonando o Repositório na Máquina

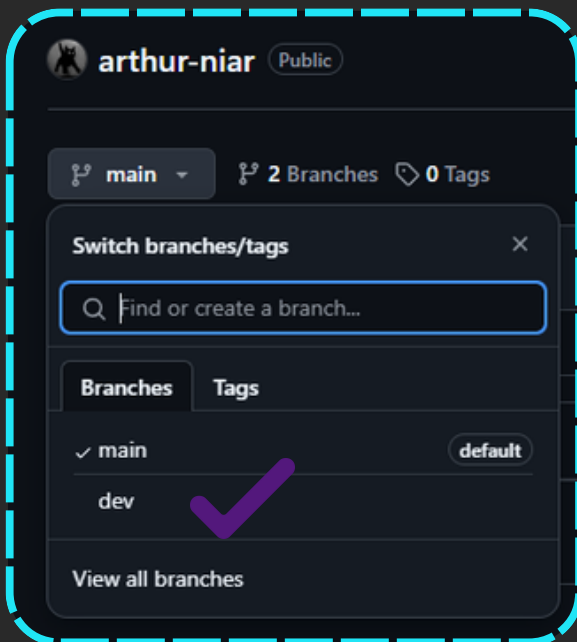
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Usuario\Desktop\nuvm\prints> git clone https://github.com/arthur-niar/arthur-niar.git
```

- Criando a branch Dev

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Usuario\Desktop\nuvm\prints> git checkout -b dev
Switched to a new branch 'dev'
PS C:\Users\Usuario\Desktop\nuvm\prints>
```

- Mandando a branch Dev para o repositório (Push)

```
PS C:\Users\Usuario\Desktop\nuvm\prints> git push origin dev
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'dev' on GitHub by visiting:
remote:   https://github.com/arthur-niar/arthur-niar/pull/new/dev
remote:
To https://github.com/arthur-niar/arthur-niar.git
 * [new branch]      dev -> dev
PS C:\Users\Usuario\Desktop\nuvm\prints>
```



- Uma boa cola nunca matou ninguém...

git cheat sheet

learn more about git the simple way at rogerdudler.github.com/git-guide/
cheat sheet created by Nina Jaeschke of ninagrailk.com

create & clone

create new repository `git init`
clone local repository `git clone /path/to/repository`
clone remote repository `git clone username@host:/path/to/repository`

add & remove

add changes to INDEX `git add <filename>`
add all changes to INDEX `git add *`
remove/delete `git rm <filename>`

commit & synchronize

commit changes `git commit -m "Commit message"`
push changes to remote repository `git push origin master`
connect local repository to remote repository `git remote add origin <server>`
update local repository with remote changes `git pull`

branches

create new branch `git checkout -b <branch>`
e.g. `git checkout -b feature_x`
switch to master branch `git checkout master`
delete branch `git branch -d <branch>`
push branch to remote repository `git push origin <branch>`

merge

merge changes from another branch `git merge <branch>`
view changes between two branches `git diff <source_branch> <target_branch>`
e.g. `git diff feature_x feature_y`

tagging

create tag `git tag <tag> <commit ID>`
e.g. `git tag 1.0.0 1b2e1d63ff`
get commit IDs `git log`

restore

replace working copy with latest from HEAD `git checkout -- <filename>`



Tip

Want a simple but powerful
git-client for your mac?
Try Tower: www.git-tower.com/

- Fonte: <http://www.genautica.com/>

DESENVOLVIMENTO

Realmente botando a mão na massa.

O processo de criar um jogo é algo que vai muito além de ter uma ideia e botar em prática. Ter noção das suas limitações, o que você pode e não pode fazer, o processo técnico e criativo, são partes essenciais do passo-a-passo do desenvolvedor.

Como um bom exemplo, vamos utilizar o vídeo de **Antarsoft**, desenvolvedor *indie* e famoso por seus tutoriais de jogos de plataforma 2D.

O processo de desenvolvimento é longo demais para demonstrar todo num livro como este, mas vou mostrar todos os detalhes mais importantes e como publicar ele utilizando as ferramentas utilizadas.



DESENVOLVIMENTO

Realmente botando a mão na massa.

O processo de criar um jogo é algo que vai muito além de ter uma ideia e botar em prática. Ter noção das suas limitações, o que você pode e não pode fazer, o processo técnico e criativo, são partes essenciais do passo-a-passo do desenvolvedor.

Como um bom exemplo, vamos utilizar o vídeo de **Antarsoft**, desenvolvedor *indie* e famoso por seus tutoriais de jogos de plataforma 2D.

O processo de desenvolvimento é longo demais para demonstrar todo num livro como este, mas vou mostrar todos os detalhes mais importantes e como publicar ele utilizando as ferramentas utilizadas.



DESENVOLVIMENTO

Utilizando as ferramentas.

O GitHub será principalmente utilizado para salvar as versões estáveis do seu jogo, códigos a parte e funcionalidades que ainda não foram para a Unity e principalmente onde pode ocorrer compilações e deploys mais complexos futuramente.

```
PS C:\Users\Usuario\Desktop\nuvem\prints> git add versaoestavel.unity
```

Além disso, utilizando o NPM, gerenciador de pacotes padrão do Node.js, usado para instalar, compartilhar e gerenciar bibliotecas e ferramentas, você pode utilizar diversas ferramentas para testar, corrigir e até facilitar o deploy do seu código.

sonarlint

- SonarLint (Linter para C#)
- Prettier (Formatar código)
- StyleCop (Formatar código)



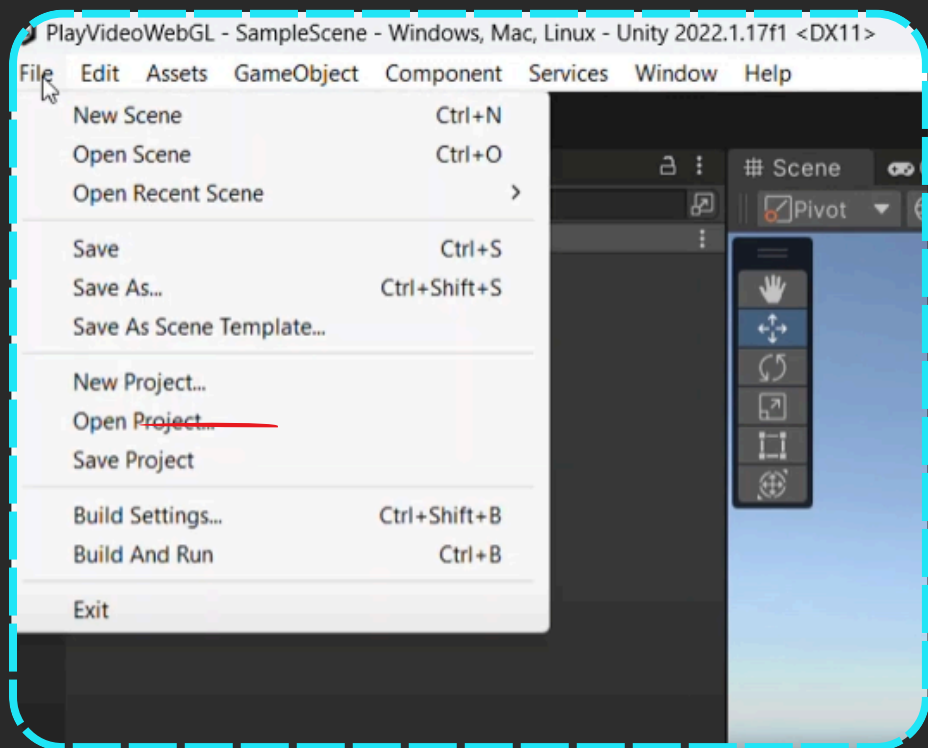
DEPLOY

WebGL e Deploy no Navegador

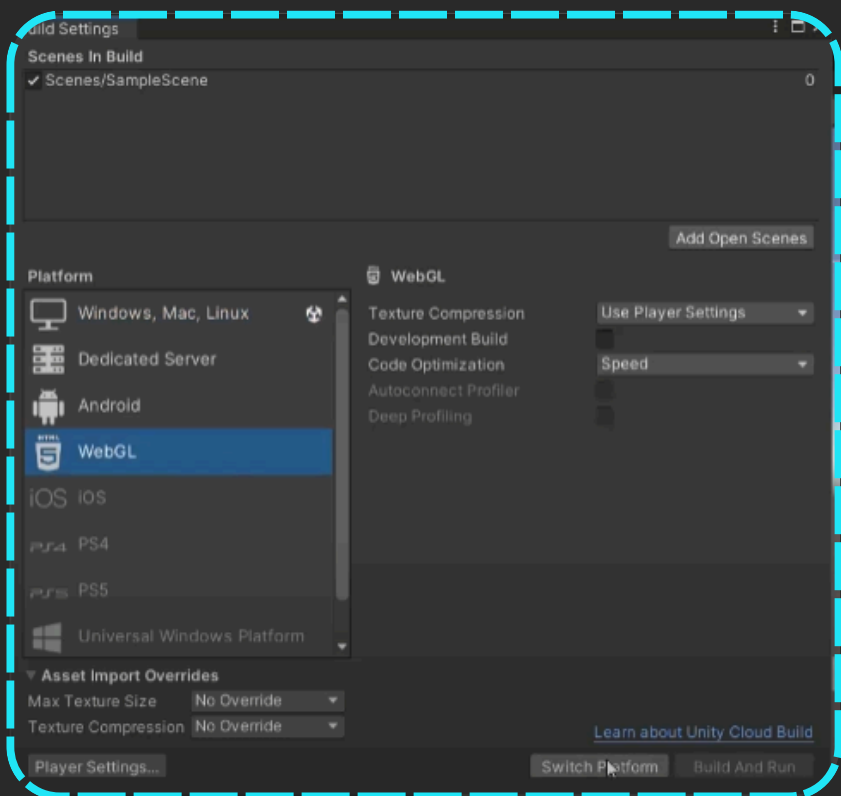
O WebGL será utilizado para transportar e compilar o projeto em um navegador para um teste simples e jogável para testes ou até uma *build* final.

Estaremos utilizando o vídeo de **Max O'Didily** como exemplo para o deploy utilizando WebGL e Github.

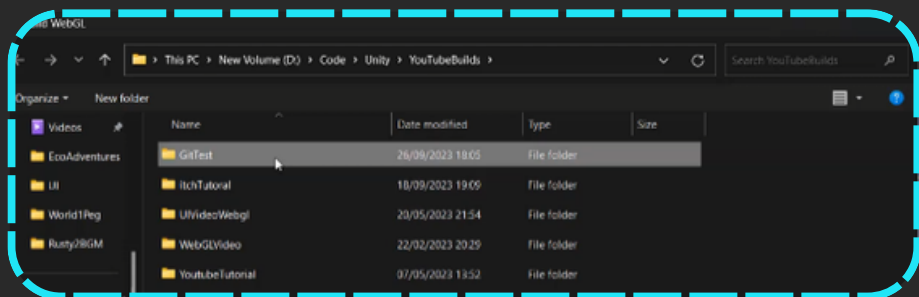
- **Na Unity, clique em File e depois em Build Settings.**



- Após clicar em Build Settings, mude a opção de Plataforma para WebGL e clique em Switch Platform.



- Depois disso, clique em Build, e a Unity pedirá uma pasta. Crie uma pasta no seu diretório preferido chamada GitTest ou qualquer outra coisa que você reconheça como Git. Selecione a pasta.



- O aplicativo vai compilar o seu projeto na pasta selecionada e criar uma versão estável (se seu jogo funcionar).
- Crie um repositório público para colocar o projeto.

Required fields are marked with an asterisk (*).

Repository template

No template ▾

Start your repository with a template repository's contents.

Owner * maxodidilly ▾ **Repository name *** WebGLTestGit

WebGLTestGit is available.

Great repository names are short and memorable. Need inspiration? How about [literate-bassoon](#)?

Description (optional)

☒ **Public**
Anyone on the internet can see this repository. You choose who can commit.

☐ **Private**
You choose who can see and commit to this repository.

Initialize this repository with:

☐ **Add a README file**
This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

gitignore template: None ▾

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

License: None ▾

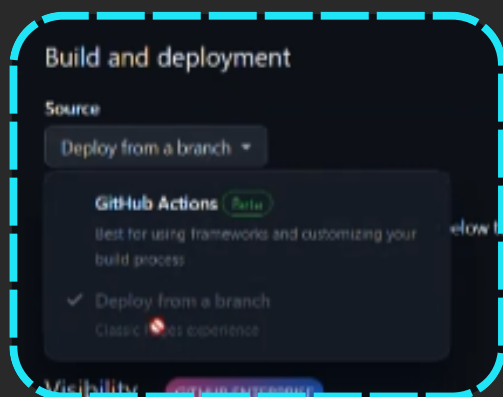
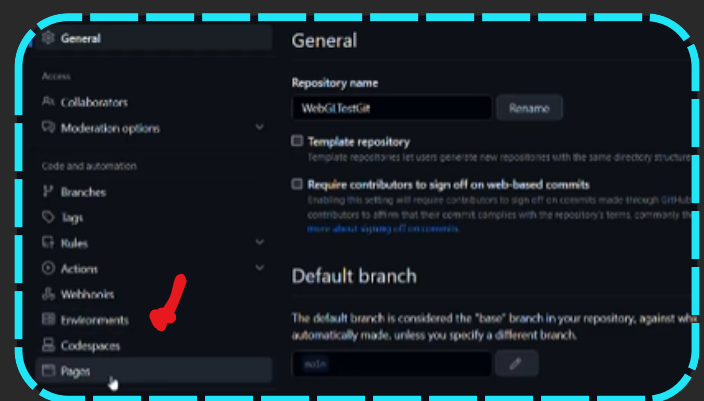
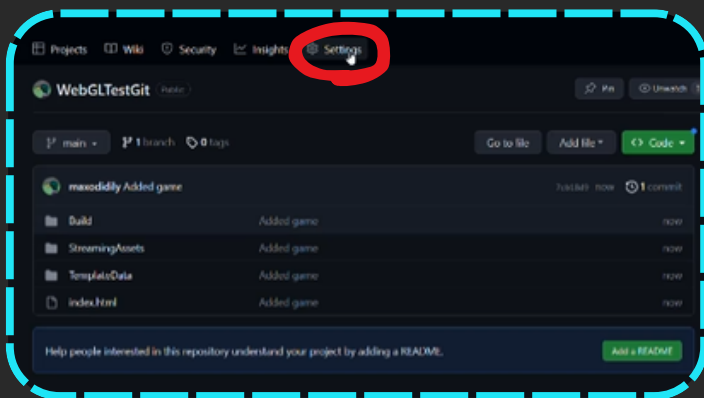
A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

🔔 You are creating a public repository in your personal account.

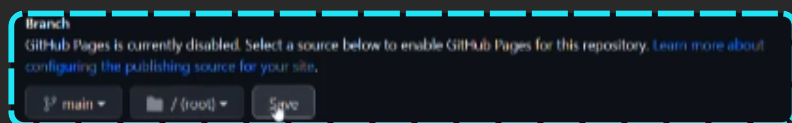
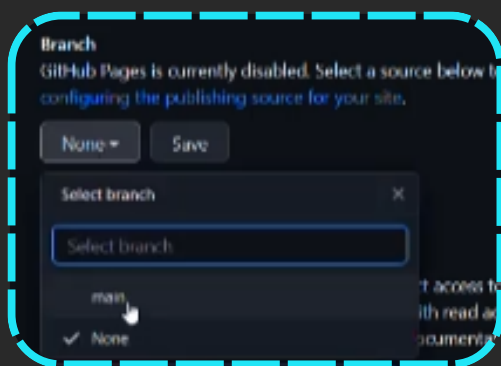
Create repository

- Coloque o jogo dentro do repositório pela pasta criada (não utilizarei o exemplo do vídeo pois ele não utilizou o terminal.)
- Ex.:
- `git add ./GitTest/`
- `git commit -m "first commit"`
- `git push origin main`

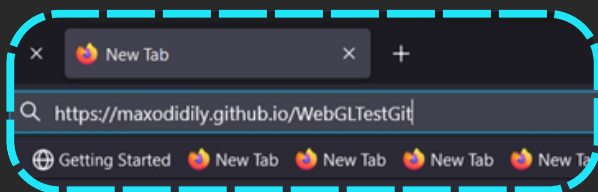
- Siga os passos abaixo para habilitar o deploy.



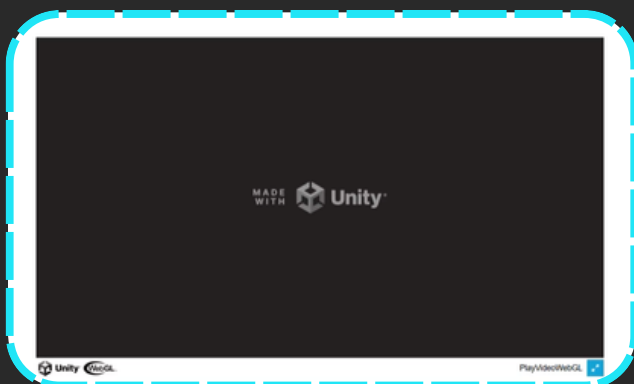
- Habilite a branch que o seu código está alocado.



- Obs: deixar o /(root) e clicar em salvar.



- Após isso, abra uma nova aba do seu navegador com o nome do repositório com esta alteração:
- [https://\[usuariogithub\].github.io/\[Repositorio\]](https://[usuariogithub].github.io/[Repositorio])
- Dessa maneira, você vai utilizar o WebGL próprio do Github para dar deploy no seu aplicativo.



- Fonte: Max O'Didily

CONCLUSÃO

O que aprendemos?

O desenvolvimento de jogos, embora seja um processo árduo e complexo, pode ser significativamente aprimorado pelo ambiente e pelas ferramentas que utilizamos durante sua criação. Cada etapa do processo, desde a concepção inicial até a entrega do produto final, é influenciada pelas decisões que tomamos e pelos recursos que escolhemos. Um ambiente bem estruturado pode tornar mais eficiente a identificação das necessidades do projeto, a aprendizagem de novas habilidades e a aplicação prática de conhecimentos para testar, corrigir e "deployar" nossa aplicação.

Além disso, o ambiente de desenvolvimento desempenha um papel crucial na colaboração da equipe, na organização do código e no aproveitamento de tecnologias modernas para simplificar tarefas desafiadoras. Quando nos situamos adequadamente no meio em que criamos o software, ganhamos a capacidade de superar obstáculos, integrar feedback rapidamente e, sobretudo, oferecer uma experiência de jogo envolvente e de qualidade. Assim, o ambiente de trabalho e as ferramentas escolhidas não são apenas suportes, mas elementos-chave que moldam todo o processo de criação.